

Bioinformatics command cheatsheet — your first week in the terminal

How to read this sheet	2
Your first five minutes	2
Part A — How to read a command	4
A1 · The parts of a command, and their names	4
A2 · Two real commands, taken apart	4
A3 · Symbols you will meet	5
A4 · Keys that save you time	5
Part B — Everyday commands	7
B1 · Where am I, and what is here?	7
B2 · Peeking inside large files	7
B3 · Copy, rename, delete — without accidents	8
B4 · Compressed files and disk space	8
B5 · Finding your Windows or Mac files	8
Part C — The workshop toolbox	9
C1 · Conda in plain English	9
C2 · Building and using the workshop environment	9
C3 · File types you will meet	10
C4 · Bioinformatics commands	10
C5 · PyMOL quick reference	11
C6 · The metagenomics pipeline	12
C7 · Installing a tool conda does not give you	15
Part D — When something goes wrong	16
D1 · Error messages, decoded	16
D2 · Getting unstuck	16
D3 · Mini glossary	16

A pocket guide for people who have never typed a command before. Read it with the terminal open beside you: find the row that matches what you want to do, type what is in the middle column, then check the screen against the last column.

Filenames such as `reads.fastq.gz` are stand-ins — use your own. Most of what follows is single commands for learning one tool at a time; section C6 strings them into the metagenomics workflow in the order the workshop runs it. Either way these are teaching commands, not a validated production pipeline.

Typing a command cannot break your computer. The two commands that can lose work are `rm` (delete) and `>` (overwrite); both are flagged where they appear.

How to read this sheet

You see	It means
A box with a blue bar on the left	Something you type, then press Enter
A box with a grey bar on the left	What your screen shows back; yours may differ slightly
text in this font	Type it exactly as printed
<like_this>	A placeholder — replace it, angle brackets and all
\$ at the start of a line	The computer's prompt. Never type the prompt itself

- Run **one line at a time**. Press Enter, wait for the prompt to come back, then type the next line.
- **Silence usually means success**. Most Unix commands print nothing when they work.
- **Capitals and spaces matter**. `Reads.fasta` and `reads.fasta` are two different files.
- Press **Tab** after the first few letters of a name and the computer finishes it for you. This prevents most typing mistakes.
- Nothing here uploads your data. Every command runs on your own machine, except `fetch` in PyMOL, which downloads a public structure.

Three different places you can type

Beginners lose the most time by typing the right command in the wrong window.

Where	Its prompt looks like	What belongs there
System terminal — Ubuntu (WSL) on Windows, Terminal on Mac	<code>you@laptop:~\$</code>	Everything in Parts A–C except the PyMOL table: <code>cd</code> , <code>ls</code> , <code>conda</code> , <code>seqkit</code> , <code>samtools</code> , and starting <code>pymol</code>
PyMOL prompt — appears after you run <code>pymol</code>	<code>PyMOL></code>	Only the commands in section C5: <code>fab</code> , <code>load</code> , <code>show</code> , <code>color</code> , <code>save</code>
Windows PowerShell	<code>PS C:\></code>	Only <code>wsl --install</code> during setup. The workshop tools do not run here

On Windows the workshop runs inside **Ubuntu (WSL)**. The bioinformatics tools come from `bioconda`, which builds software for Linux and macOS only, so they cannot run in PowerShell or in the native Windows Miniforge Prompt.

Your first five minutes

1. **Open the right terminal**. Windows: Start menu → **Ubuntu**. Mac: Spotlight (`Cmd+Space`) → **Terminal**.
2. **Switch on the workshop toolbox**.

```
conda activate protein_modeling
```

Your prompt should now begin with the environment name in brackets:

```
(protein_modeling) you@laptop:~$
```

3. **Ask where you are and what is around you**.

```
pwd
ls -lh
```

```
/home/you
total 8.0K
drwxr-xr-x 2 you you 4.0K Oct  7 09:14 workshop
```

4. Move into your workshop folder and make somewhere to put results.

```
cd ~/workshop
mkdir -p results
pwd
```

5. When you are finished for the day, put the toolbox away and close the window:

```
conda deactivate
```

If step 2 answers conda: command not found — close the terminal, open a new one, and try again; conda only becomes available in terminals opened after it was installed. If it still fails, redo Module 3 of the setup manual.

```
program subcommand --option value -f input.ext > output.ext
|               |           |         |             |
|               |           |         |             └─ send the result into a file
|               |           |         └─ the file you want the program to read
|               |           └─ short flag (one dash, one letter)
|               └─ the value that belongs to the option
|               └─ long option (two dashes, a whole word)
└─ which job of that program to do
└─ the program you are running
```

```
pymol -cq -d "fab ACDE, peptide; save peptide.pdb, peptide; quit"
```

```
pymol -cq -d "fab ACDE, peptide; save peptide.pdb, peptide; quit"
├── pymol: the program
│   ├── -c: no window; -q: start quietly
│   ├── -d: run the quoted commands
│   │   ├── fab: build a peptide
│   │   │   ├── ACDE: a four-letter toy amino-acid sequence
│   │   │   └── peptide: the name we gave the object
│   │   └── save peptide.pdb, peptide:
│   │       ├── save: write peptide.pdb from the object
│   │       └── peptide: the name we gave the object
│   └── quit: close PyMOL
```

The quotation marks hold the three PyMOL commands together as a single argument, and the semicolons separate them. In `save peptide.pdb, peptide`, the first name is the file to write and the second is the object to write out. The file lands in whatever folder you were in, so check `pwd` first. Notice that this command has **no subcommand and no long option** — not every program uses every part of the diagram. `fab ACDE` builds a toy model for practice, never experimental data.

A3 • Symbols you will meet

Symbol	Say it as	What it does	Example
~	"home"	Your personal folder	<code>cd ~/workshop</code>
.	"here"	The folder you are in right now	<code>cp file.fasta .</code>
..	"up one"	The folder above this one	<code>cd ..</code>
/	"slash"	Separates folders in a path	<code>~/workshop/results</code>
*	"star" or "any"	Stands for any characters — *.fasta means every FASTA file here	<code>ls *.fastq.gz</code>
"..."	"quotes"	Keeps a name with spaces in one piece	<code>ls "my data"</code>
>	"into"	Writes the result into a file, erasing that file first	<code>seqkit stats reads.fastq.gz > stats.txt</code>
>>	"add into"	Adds the result to the end of a file	<code>echo done >> run.log</code>
2>	"errors into"	Saves the error messages instead of the result	<code>bwa index ref.fasta 2> errors.txt</code>
	"pipe"	Feeds one command's result straight into the next	<code>zcat reads.fastq.gz less</code>
\ at line end	"continued"	The command carries on across the next line	see the conda create block in C2
#	"note"	The rest of the line is a comment, not a command	<code>mkdir results # outputs go here</code>

Two words appear in error messages and in tool documentation: **stdout** is the normal result a command prints, and **stderr** is its complaints. `>` captures the first, `2>` the second. That is the whole distinction.

A4 • Keys that save you time

Key	What it does
Tab	Completes a file or command name you have started typing
Up / Down arrow	Brings back the previous / next command so you can reuse or fix it
Ctrl+C	Stops the command that is running right now

Key	What it does
Ctrl+R	Searches your command history — type a few letters of an old command
Ctrl+L	Clears the screen (same as typing <code>clear</code>)
Ctrl+A / Ctrl+E	Jumps to the start / end of the line you are typing
q	Leaves <code>less</code> , <code>man</code> , and anything else that fills the screen and waits

If the prompt does not come back and nothing seems to be happening, the command is probably still working — large files take time. `Ctrl+C` cancels it if you need the prompt back.

Part B — Everyday commands

B1 • Where am I, and what is here?

What you want	Type this	What you get
Know which folder you are in	<code>pwd</code>	A path such as <code>/home/you/workshop</code>
List what is here, with sizes	<code>ls -lh</code>	One line per file: permissions, size, date, name
Include hidden files	<code>ls -a</code>	Also shows names beginning with <code>.</code>
Go into a folder	<code>cd results</code>	The prompt changes; <code>pwd</code> confirms it
Go back up one level	<code>cd ..</code>	You are in the parent folder
Go straight home	<code>cd ~</code> or <code>cd</code>	Back to <code>/home/you</code>
Make a folder (and its parents)	<code>mkdir -p results/figures</code>	Nothing printed — that means it worked
Count the lines in a file	<code>wc -l samples.txt</code>	<code>42 samples.txt</code>
Find a word inside a file	<code>grep 'BRCA1' annotations.tsv</code>	Every matching line
Find files by name	<code>find . -name "*.fasta"</code>	Paths of matching files, including subfolders
See where a tool is installed	<code>which samtools</code>	A path inside your conda environment
Check free disk space	<code>df -h .</code>	Size, used, and available for this drive
Read the full manual for a command	<code>man ls</code>	A scrollable manual; press <code>q</code> to leave
Repeat something you ran earlier	<code>history</code>	A numbered list of recent commands

Get into the habit of running `pwd` before you create, overwrite, or delete anything. Almost every beginner accident is a correct command run in the wrong folder.

B2 • Peeking inside large files

Sequence files can be millions of lines long. Opening one whole can freeze a text editor, so look at a piece of it instead.

What you want	Type this	What you get
The first 20 lines	<code>head -n 20 annotations.tsv</code>	Enough to see the column headers
The last 20 lines	<code>tail -n 20 run.log</code>	The most recent output of a long job
Watch a log as it grows	<code>tail -f run.log</code>	New lines appear live; <code>Ctrl+C</code> stops watching
Scroll through a text file	<code>less sequences.fasta</code>	Arrow keys scroll, <code>/word</code> searches, <code>q</code> quits
Print a small file	<code>cat notes.txt</code>	The whole file at once — only for short files
Look inside a <code>.gz</code> without unpacking it	<code>gunzip -c reads.fastq.gz head -n 8</code>	The first two FASTQ records
Count the sequences in a FASTA	<code>grep -c '>' reference.fasta</code>	A single number
Pick out columns of a table	<code>cut -f1,2 annotations.tsv</code>	Columns 1 and 2 only
Sort lines	<code>sort samples.txt</code>	The same lines, alphabetically
Count how often each line appears	<code>sort ids.txt uniq -c</code>	A count in front of each unique line

`uniq` only collapses neighbouring duplicates, which is why it is almost always used after `sort`.

B3 • Copy, rename, delete — without accidents

What you want	Type this	Watch out for
Copy a file into a folder	<code>cp input.fasta results/</code>	If a file of that name is there, it is replaced
Copy a whole folder	<code>cp -r data/ backup/</code>	-r means “and everything inside”
Rename or move	<code>mv old.fasta new.fasta</code>	Same command for both jobs
Delete, asking first	<code>rm -i old.fasta</code>	Answer y or n. There is no recycle bin
Delete an empty folder	<code>rmdir emptydir</code>	Refuses if anything is inside — that is a safety feature
Make a shortcut to a big file	<code>ln -s /mnt/c/PIPELINE/Ologist_Workshop/Datasets data</code>	Points at the original; does not copy it
Make a script runnable	<code>chmod +x setup_conda_envs.sh</code>	Needed once before <code>./setup_conda_envs.sh</code>
Run a script in this folder	<code>./setup_conda_envs.sh</code>	The <code>./</code> means “the one right here”

The one genuinely dangerous command. `rm` deletes immediately and permanently, and `rm -r` deletes a folder and everything in it. Use `rm -i` while you are learning, check `pwd` first, and never run `rm` with `*` until you have listed the same pattern with `ls` and seen exactly which files match.

B4 • Compressed files and disk space

Sequencing data arrives compressed, usually as `.gz` or `.tar.gz`. Most bioinformatics tools read `.gz` directly, so you often do not need to unpack anything.

What you want	Type this
Unpack a <code>.tar.gz</code> download	<code>tar -xzf data.tar.gz</code>
Bundle a folder for sharing	<code>tar -czf out.tar.gz folder/</code>
Decompress but keep the original	<code>gunzip -k reads.fastq.gz</code>
Compress but keep the original	<code>gzip -k reads.fastq</code>
Measure a folder's total size	<code>du -sh results/</code>

B5 • Finding your Windows or Mac files

What you want	Type this
Reach your Windows C: drive from Ubuntu	<code>cd /mnt/c/Users/<YourName>/Downloads</code>
Open the current folder in Windows File Explorer	<code>explorer.exe .</code>
Open the current folder in Mac Finder	<code>open .</code>
Edit a text file without leaving the terminal	<code>nano notes.txt</code> — Ctrl+O saves, Ctrl+X exits

Workshop data lives at `/mnt/c/PIPELINE/Ologist_Workshop/Datasets`, which Windows displays as `C:\PIPELINE\Ologist_Workshop\Datasets`. Two rules follow from this: Ubuntu writes paths with forward slashes, and new Linux work belongs in your fast Linux home (`~/workshop`), reaching across to `/mnt/c/` only for files that already live on the Windows side.

Part C — The workshop toolbox

C1 • Conda in plain English

Conda installs scientific software for you, including the parts that are awkward to compile by hand. An **environment** is a named, self-contained set of that software. The workshop's environment is called `protein_modeling`, and activating it is what makes `seqkit`, `samtools`, `pymol`, and the rest exist as commands.

Three ideas are worth holding on to:

- **Activating is not installing.** You install once; you activate in every new terminal window.
- **(base) is not your environment.** Conda's own starting environment is called `base`; keep project packages out of it.
- **Environments are disposable, your data is not.** If an environment breaks, delete and rebuild it. Keep data and results in `~/workshop`, never inside the environment folder.

Most of this sheet assumes `protein_modeling`. The metagenomics tools in C6 are the exception: they sit in a second environment, `metagenomics_env`, and only one environment is active at a time. If a command exists but is “not found”, check which environment your prompt names before anything else.

A **channel** is simply where conda downloads from. This workshop uses `conda-forge` (general scientific software) and `bioconda` (bioinformatics tools). Bioconda builds for Linux and macOS only — the reason Windows users work inside WSL Ubuntu.

C2 • Building and using the workshop environment

The whole workshop runs in **one** environment holding Biopython, pandas, PyMOL, the bioinformatics tools, and the docking engine together, so you never switch environments mid-exercise. Build it once:

```
conda --version
bash setup_conda_envs.sh
conda activate protein_modeling
```

Run `bash setup_conda_envs.sh` from inside the `Ologist_Workshop` project folder. The script creates the environment, and skips the work if it already exists. What it runs, in effect, is this single command — use it directly if you do not have the project folder. Type `y` when conda asks for confirmation, and expect several minutes of downloading:

```
conda create -n protein_modeling \
  -c conda-forge -c bioconda python=3.11 \
  biopython pandas pymol-open-source \
  fastqc seqkit bwa samtools bcftools blast \
  vina meeko rdkit
```

Check the result, and leave the environment when you are done for the day:

```
conda info --envs
python -c "import Bio, pandas, pymol; print('biopython', Bio.__version__, 'pandas',
      pandas.__version__)"
conda deactivate
```

A healthy environment answers like this — the `*` marks the active one, and your version numbers will differ:

```
# conda environments:
#
base                /home/you/miniforge3
protein_modeling    * /home/you/miniforge3/envs/protein_modeling

biopython 1.85 pandas 2.2.3
```

Task	Command
List everything installed in the active environment	<code>conda list</code>
Check one package is there	<code>conda list seqkit</code>
Add a package to the active environment	<code>conda install -c conda-forge -c bioconda pdbfixer</code>
Write down the exact recipe	<code>conda env export -n protein_modeling > environment.yml</code>
Rebuild from that recipe on a compatible machine	<code>conda env create -f environment.yml</code>
Delete an environment you are not in (permanent)	<code>conda env remove -n old-env</code>
See which environment is active	<code>conda info --envs</code> , or read your prompt
Switch to the metagenomics toolbox	<code>conda activate metagenomics_env</code>

Never run `conda` with `sudo`: it installs into your own home folder and does not need administrator rights, and using `sudo` creates files you can no longer edit. An exported `environment.yml` can contain builds specific to one operating system and a local `prefix:` line, so read it before sharing it across machines.

C3 • File types you will meet

Extension	What is inside	How to look at it
<code>.fasta</code> , <code>.fa</code> , <code>.faa</code> , <code>.fna</code>	Sequences, each starting with a > name line	<code>head -n 4 file.fasta</code>
<code>.fastq</code> , <code>.fastq.gz</code>	Sequencing reads plus a quality score per base	<code>seqkit stats file.fastq.gz</code>
<code>.gz</code>	Any of the above, compressed	Most tools read it as-is
<code>.sam</code> / <code>.bam</code>	Reads after alignment to a reference; BAM is the compressed form	<code>samtools view file.bam head</code>
<code>.vcf</code> , <code>.vcf.gz</code>	Variants — positions where a sample differs from the reference	<code>bcftools view -h file.vcf.gz</code>
<code>.pdb</code> , <code>.cif</code>	3-D atomic coordinates of a structure	<code>load file.pdb</code> in PyMOL
<code>.pse</code>	A saved PyMOL session: objects, colours, and view	<code>load file.pse</code> in PyMOL
<code>.gfa</code>	An assembly graph: contigs plus how they connect	<code>open file.gfa</code> in Bandage
<code>.gff3</code> , <code>.gbff</code>	Annotation — what was found, and where on the contig	<code>head -n 20 file.gff3</code>
<code>.tsv</code>	A table, one feature or hit per line	<code>cut -f1,2 file.tsv head</code>

C4 • Bioinformatics commands

Run these in the system terminal with `protein_modeling` active. Check the obvious things before interpreting any result: that the input is the file you meant, that read pairs match, and that the reference assembly is the right one. Remember that `*.fastq.gz` means every matching file in the current folder.

What you want	Type this
Summarise sequences: how many, how long	<code>seqkit stats *.fastq.gz</code>
Make a folder for the output first	<code>mkdir -p results</code>
Make a read-quality report into it	<code>fastqc -o results reads.fastq.gz</code>
Pull one named record out of a FASTA	<code>seqkit grep -n -p geneA reference.fasta > geneA.fasta</code>

What you want	Type this
Prepare a reference before aligning to it	<code>bwa index reference.fasta</code>
Index a sorted BAM so tools can jump within it	<code>samtools index sample.sorted.bam</code>
Ask how many reads aligned	<code>samtools flagstat sample.sorted.bam</code>
Look at one region of an indexed BAM	<code>samtools view sample.sorted.bam chr1:1000-2000</code>
Read a VCF's header	<code>bcftools view -h variants.vcf.gz</code>
Pull chosen fields out of a VCF	<code>bcftools query -f '%CHROM\t%POS\t%REF\t%ALT\n' variants.vcf.gz</code>

`seqkit stats` is the fastest sanity check you have, and a good habit before and after every step:

```
file          format type num_seqs  sum_len  min_len  avg_len  max_len
reads_R1.fastq.gz FASTQ  DNA    500,000  75,000,000  150     150     150
```

Align paired reads to a reference and sort the result in one go. Run `bwa index` first, and make sure `R1` and `R2` are the matching mates of one sample:

```
bwa mem reference.fasta R1.fastq.gz R2.fastq.gz |
samtools sort -o sample.sorted.bam -
```

The final `-` is not a typo: it tells `samtools sort` to read from the pipe rather than from a file.

Compare two sequences you already have on disk. This is a local comparison, not a search of NCBI's online database:

```
blastn -query query.fasta -subject reference.fasta -outfmt 6 > hits.tsv
```

C5 • PyMOL quick reference

Start PyMOL from the system terminal with the environment active:

```
pymol
```

A window opens and the terminal switches to the `PyMOL>` prompt. **Everything in this table is typed at `PyMOL>`, one line at a time** — these are not system commands.

What you want	Type this at PyMOL>
Build a toy peptide to practise on	<code>fab ACDE, peptide</code>
Open a structure file from this folder	<code>load protein.pdb, protein</code>
Download a published structure (needs internet)	<code>fetch lubq, async=0</code>
Clear the display of one object	<code>hide everything, protein</code>
Show the fold as a ribbon	<code>show cartoon, protein</code>
Show individual atoms and bonds	<code>show sticks, peptide</code>
Colour it	<code>color cyan, protein</code>
Fill the window with one object	<code>zoom protein</code>
Name a region so you can work on it	<code>select site, protein and chain A and resi 10-20</code>
Highlight that region	<code>show sticks, site</code>
Colour the highlighted region	<code>color yellow, site</code>
Lay one structure over another	<code>align mobile, target</code>
Save the whole scene to carry on later	<code>save scene.pse</code>
Save just the coordinates	<code>save protein.pdb, protein</code>

What you want	Type this at PyMOL>
Read the help for any command	<code>help color</code>
Leave PyMOL	<code>quit</code>

Making a figure takes three commands, in this order — a white background, a slow high-quality render, then the file itself:

```
bg_color white
ray 1200, 900
png figure.png
```

Files are written into whatever folder you started `pymol` from, unless you give a full path. A `.pse` keeps your editable scene — objects, colours, and camera; a `.pdb` keeps only coordinates. `fab ACDE` is a toy model for practice, and any real PDB entry you `fetch` should be cited in your report. To run PyMOL without opening a window at all, use the one-line form in section A2.

C6 • The metagenomics pipeline

These tools live in a **second** environment, `metagenomics_env`, built in Module 7 of the setup manual. The assemblers are large and pull in their own dependency sets, so they are kept out of `protein_modeling` rather than fighting it. Switch toolbox with `conda activate metagenomics_env`, and go back with `conda deactivate` followed by `conda activate protein_modeling`. A `command not found` here is nearly always the other environment being active.

The pipeline runs one way, and each step reads the step before it:

```
FASTQ → QC → cleaning → assembly → evaluation → graph → annotation → screening
```

Stage	Tool	Type this	Reads	Writes
Short-read QC	FastQC	<code>fastqc</code>	short FASTQ	HTML report
Long-read QC	NanoStat	<code>NanoStat</code>	long FASTQ	a table of statistics
Short-read cleaning	<code>fastp</code>	<code>fastp</code>	R1 + R2	clean R1 + R2, HTML/JSON
Long-read cleaning	<code>fastplong</code>	<code>fastplong</code>	long FASTQ	clean long reads, HTML/JSON
Short-read assembly	metaSPAdes	<code>metaspades.py</code>	clean R1 + R2	<code>contigs.fasta</code> , <code>.gfa</code>
Long-read assembly	MetaFlye	<code>flye</code>	clean long reads	<code>assembly.fasta</code> , <code>.gfa</code>
Hybrid assembly	OPERA-MS	<code>perl OPERA-MS.pl</code>	short + long	hybrid contigs
Evaluation	metaQUAST	<code>metaquast</code>	FASTA	QC report, metrics
Graph	Bandage	<code>Bandage</code>	<code>.gfa</code>	a picture you read yourself
Annotation	Bakta	<code>bakta</code>	FASTA	TSV, GFF3, GenBank, proteins
AMR	AMRFinderPlus	<code>amrfinder</code>	FASTA	AMR hit table
AMR + virulence	ABRicate	<code>abricate</code>	FASTA	screening table

Never write cleaned reads over raw ones. Raw FASTQ is the one thing in the project you cannot regenerate. Cleaning writes to a *different* folder, always.

Set up the folders once

```
mkdir -p ~/workshop/metagenomics/{raw_data/{short_reads,long_reads},cleaned_data,\
qc/{raw,cleaned},assembly/{metaspades,metaflye,opera_ms},evaluation,annotation,screening}
cd ~/workshop/metagenomics
```

Every command below assumes you are in that folder, so the short paths work as printed.

Quality control, before and after

```
fastqc raw_data/short_reads/short_R1.fastq.gz \
       raw_data/short_reads/short_R2.fastq.gz -o qc/raw/

NanoStat --fastq raw_data/long_reads/long_reads.fastq.gz > qc/raw/nanostat.txt
```

Open the FastQC .html in a browser (explorer.exe qc/raw on Windows, open qc/raw on a Mac). Read per-base quality, adapter content, sequence-length distribution and overrepresented sequences. NanoStat prints its long-read statistics straight to the file you redirected it into.

Run both tools again on the cleaned files afterwards, writing into qc/cleaned/, so you can see what cleaning actually did.

Cleaning reads

```
fastp \
  -i raw_data/short_reads/short_R1.fastq.gz \
  -I raw_data/short_reads/short_R2.fastq.gz \
  -o cleaned_data/short_R1.clean.fastq.gz \
  -O cleaned_data/short_R2.clean.fastq.gz \
  --cut_right --cut_right_window_size 4 --cut_right_mean_quality 20 \
  --qualified_quality_phred 20 --unqualified_percent_limit 30 \
  --length_required 50 --thread 4 \
  --html qc/fastp.html --json qc/fastp.json
```

Lower-case -i / -o are read 1 in and out; capital -I / -O are read 2. What the rest of it asks for:

Option	What it does
--cut_right	Slides a window along the read from the 5' end and cuts where the window's average quality drops
--cut_right_window_size 4	That window is 4 bases wide
--cut_right_mean_quality 20	Cut when the window averages below Q20
--qualified_quality_phred 20	A base counts as "good" at Q20 or above
--unqualified_percent_limit 30	Drop the read if more than 30% of its bases are below that
--length_required 50	Drop reads shorter than 50 bases after trimming
--thread 4	Use 4 CPU threads

Long reads get their own tool, with the defaults as a sensible start:

```
fastplong -i raw_data/long_reads/long_reads.fastq.gz \
          -o cleaned_data/long_reads.clean.fastq.gz
```

Assembly

```
metaspades.py -1 cleaned_data/short_R1.clean.fastq.gz \
              -2 cleaned_data/short_R2.clean.fastq.gz \
              -o assembly/metaspades

flye --nano-raw cleaned_data/long_reads.clean.fastq.gz --meta \
     --out-dir assembly/metaflye --threads 8
```

If your install gives you spades.py but not metaspades.py, the equivalent is spades.py --meta with the same -1 / -2 / -o. Assembly is the slowest step of the day and the one most likely to run out of memory: a Killed message means the machine, not your command, gave up — ask the instructor for a smaller input or the workshop server.

Evaluation and the assembly graph

```
metaquast assembly/metaspades/contigs.fasta -o evaluation/metaspades
metaquast assembly/metaflye/assembly.fasta -o evaluation/metaflye

ls -lh assembly/metaspades/ assembly/metaflye/
Bandage
```

If `metaquast` is not found, the same tool is there as `metaquast.py` — metaQUAST ships inside the `quast` package and the name depends on the build.

metaQUAST writes a report you read in a browser. The six numbers you compare between assemblies:

Metric	What it means
Number of contigs	How many separate pieces the assembler produced. Fewer, longer pieces is usually better
Total length	All contigs added together. Compare it against the genome sizes you expect
Largest contig	The single longest piece
N50	Half of the assembly sits in contigs at least this long
L50	How many contigs it takes to reach that half. Low is good
GC content	Percentage of G and C bases — a rough identity check on the organisms present

In Bandage, use **File → Load graph** on the `.gfa` in either assembly folder, then **Draw graph**. Look for the largest connected component, branch points, dead ends, loops, and separate components. A graph shows you structure — tangles where a repeat could not be resolved — that no single number can.

N50 alone does not pick the winner. A high N50 can come from one long misassembled contig. Weigh it against total length, how much of the expected genome you recovered, and what the graph looks like.

Annotation and screening

```
bakta --db <path/to/bakta_db> --output annotation/metaspades \
      --prefix metaspades assembly/metaspades/contigs.fasta

amrfinder --nucleotide assembly/metaspades/contigs.fasta \
          --output screening/amrfinder_metaspades.tsv

abricate --db vfdb assembly/metaspades/contigs.fasta \
          > screening/vfdb_metaspades.tsv
```

Both `bakta` and `amrfinder` need a database as well as a program, and the database has its own version. Record both: `bakta_db list` shows what is available, `amrfinder --database_version` reports the one in use, and `abricate --list` shows which ABRicate databases are set up. Two labs running the same command against different database versions can honestly get different answers.

From Bakta's output, `*.tsv` is the readable feature table, `*.gff3` is the one other tools read, and `*.faa` holds the predicted proteins. Count CDS, tRNA, rRNA and ncRNA features from the summary.

A hit in a database is sequence similarity, nothing more. It is not proof that the gene is expressed, that the organism is resistant, or that an infection would behave a certain way. And a *missing* hit is not proof of absence: fragmented assembly, low coverage, tool thresholds and database scope all hide real genes. Say “detected in this assembly, by this tool and database version” and you will never have to walk a claim back.

What to write down as you go

For every step, three things make the work reproducible: the **exact command** you ran, the **tool version** (`<tool> --version`), and the **database version** where one applies. Keep them in a plain text file as you go; `history > run_commands.txt` at the end of a session is a useful backstop, not a substitute for noting them at the time.

C7 • Installing a tool conda does not give you

You have **two** package managers, and they do different jobs. Conda installs research software into an environment, where it appears only while that environment is active. The system package manager installs general command-line utilities into the machine itself, where they work in every terminal whether an environment is active or not. On Ubuntu and WSL that is `apt`; on a Mac it is Homebrew (`brew`).

Two small utilities worth having, as a worked example. `tree` draws a folder and its contents as an indented tree, which is the quickest way to see what a pipeline actually produced. `bttop` is a live display of CPU, memory and disk, which is how you tell a long assembly apart from a hung one.

Ubuntu and WSL — `apt` needs administrator rights, so it asks for your Ubuntu password. Refresh the package list first; installing without it is the usual cause of “package not found” on a fresh system:

```
sudo apt update
sudo apt install -y tree bttop
```

Mac — Homebrew needs no `sudo`:

```
brew install tree bttop
```

If `brew: command not found`, Homebrew is not installed yet; Module 3 of the setup manual covers it. If `bttop` is not available on an older Ubuntu, `htop` is the same idea and is always there.

What you want	Type this
See a folder as a tree	<code>tree ~/workshop</code>
Keep it readable — two levels only	<code>tree -L 2 ~/workshop</code>
Include file sizes	<code>tree -L 2 -h ~/workshop</code>
Directories only	<code>tree -d ~/workshop</code>
Watch CPU and memory live	<code>bttop</code> — press q to quit
Check a tool arrived	<code>which tree, tree --version</code>
Remove something you installed	<code>sudo apt remove tree, or brew uninstall tree</code>

Three rules follow from having two managers:

- **sudo belongs to apt, never to conda.** `apt` installs system-wide and needs permission; conda installs into your own home folder and does not. `sudo conda` creates files you can no longer edit.
- **Research tools go in conda; utilities can go either way.** Put `samtools`, `seqkit` and the rest in the environment, so the version is recorded with the project. A convenience like `tree` is fine system-wide.
- **No administrator rights? Use conda instead.** On a managed laptop where `sudo` is refused, `conda install -c conda-forge tree` puts the same tool in your environment without needing permission from anyone.

Part D — When something goes wrong

D1 • Error messages, decoded

The screen says	It usually means	Try this
command not found	The environment is off, or the name is misspelled	<code>conda activate protein_modeling</code> , then retype using Tab
<code>conda: command not found</code>	This terminal was opened before conda was installed	Close the terminal, open a new one
No such file or directory	You are not in the folder you think you are in	<code>pwd</code> , then <code>ls</code> , then <code>cd</code> to the right place
Permission denied	The file is not marked runnable, or the folder is protected	<code>chmod +x script.sh</code> , or work inside <code>~/workshop</code>
Is a directory	You gave a folder where a file was expected	Add the filename to the path
No space left on device	The disk is full	<code>df -h .</code> and <code>du -sh *</code> to find what is large
Killed	The job ran out of memory — common during assembly	Use a smaller input, or move to the workshop server
A tool reports a missing or wrong database	The program installed; its database did not	<code>bakta_db list</code> , <code>amrfinder --update</code> , or <code>abricate --list</code>
Bandage will not open a window	A graphical program on a text-only setup	Use the trainer's build, or view the graph on the workshop server
Nothing at all, prompt returns	The command worked	Check with <code>ls -lh</code> that the output file exists
Pages of text flying past	Normal output, too much of it	Repeat with <code> less</code> , or send it to a file with <code>></code>

D2 • Getting unstuck

Situation	Way out
The prompt will not come back	<code>Ctrl+C</code> cancels the running command
You are stuck in a full-screen pager or manual	Press <code>q</code>
You are stuck in the nano editor	<code>Ctrl+O</code> then Enter to save, <code>Ctrl+X</code> to leave
You are at <code>PyMOL></code> and want the terminal back	Type <code>quit</code>
A command is half-typed and wrong	<code>Ctrl+C</code> starts a fresh line without running it
You cannot remember the exact command you used	<code>Ctrl+R</code> , then type a few letters of it
You do not know what a tool can do	<code><tool> --help</code> , or <code>man <tool></code> for the built-in commands

When you ask for help, copy the whole thing: the command you typed, the full error message, and the output of `pwd` and `conda info --envs`. That trio answers most questions before they are asked.

D3 • Mini glossary

Term	In one line
Terminal	The window where you type commands
Shell	The program inside that window reading what you type (usually <code>bash</code>)

Term	In one line
Prompt	The text waiting at the start of the line, such as <code>you@laptop:~\$</code>
Command	One instruction: a program plus its options and files
Flag / option	A setting on a command, written <code>-n</code> or <code>--name</code>
Argument	Anything you hand to a command, such as a filename
Path	A file's address, like <code>~/workshop/results/stats.txt</code>
Directory	A folder
Home	Your personal folder, written <code>~</code>
Environment	A named set of installed software you switch on with <code>conda activate</code>
Package	One piece of installable software, such as <code>samtools</code>
Channel	The source conda downloads packages from, such as <code>bioconda</code>
<code>apt</code> / Homebrew	The system package manager on Ubuntu / on a Mac, for tools outside conda
Pipe	<code> </code> , which feeds one command's output into the next
Redirect	<code>></code> , which writes output into a file
<code>stdout</code> / <code>stderr</code>	A command's normal output / its error messages
GUI / headless	With a window / without one
Read	One sequence as it came off the machine
Contig	A longer sequence the assembler built by overlapping reads
Assembly graph	Contigs plus how they connect, kept in a <code>.gfa</code>
N50	The length at which half the assembly sits in contigs at least that long
WSL	Windows Subsystem for Linux — real Ubuntu running inside Windows

Further help: [workshop setup manual](#) · [protein modeling and docking manual](#) · [conda user guide](#) · [PyMOL command reference](#).